

AmazonHelp AI Support Agent Report

Goal: turn messy Twitter support data into a trustworthy AI agent for a brand, then prove it with evaluation.

Brand: @AmazonHelp Dataset from Kaggle thoughtvector/customer-support-on-twitter.

LLM: Groq openai/gpt-oss-120b (free tier).

Embeddings: local all-MiniLM-L6-v2 via ChromaDB (no paid embedding API).

1. End-to-end project walkthrough

Step A - Data: `download_data.py` pulls the Kaggle CSV, `data_processing.py` keeps only AmazonHelp replies linked to the customer tweet they answered, producing `customer_text` / `brand_text` pairs plus a keyword heuristic `intent` for exploration.

Step B - Retrieval: `rag_pipeline.py` indexes ~3000 customer questions into a local ChromaDB collection and stores the historical AmazonHelp reply as metadata.

Step C - Agent (`agent.py`): for each new message, retrieve top-3 similar past issues, then call Groq to return structured JSON: `intent`, `draft_reply`, `action` (auto-handle|escalate), and `reason`. Drafts are constrained to Twitter length and grounded in retrieved brand tone (often DM / ask for order details).

Step D - Golden set: `golden_set_generator.py` samples ~200 intent-balanced examples, proposes silver labels with Groq, and leaves `human_intent` / `human_action` / `reviewed` columns for mandatory manual verification (see docs/GOLDEN_SET_METHODODOLOGY.md).

Step E - Evaluation harness: `evaluation.py` compares

(1) trivial baseline, (2) zero-shot Groq without RAG, (3) full RAG agent on intent/action accuracy,

runs an LLM-as-judge rubric on drafts, optionally measures judge vs human agreement from `data/human_reply_ratings.csv` and logs everything to MLflow.

2. Problem framing

What a "Good" AI Agent Looks Like:

- **Accurate Understanding:** It correctly identifies what the customer needs.
- **On-Brand Responses:** It writes short, empathetic replies that sound exactly like real Amazon customer service agents.
- **Smart Handoffs:** It automatically transfers the conversation to a human agent if the issue involves money, account security, or high emotion.
- **Automated Easy Fixes:** It only fully handles simple, common questions (like order status updates or FAQs).

3. Results vs baselines

Trivial: majority intent + always escalate.

Simple: zero-shot Groq, no RAG.

Agent RAG: Groq + top-3 historical pairs.

Metrics from `evaluation.py` on n=30 golden examples (LLM judge n=18; judge vs human overlap n=17)

Trivial - intent acc: 0.40 | action acc: 0.57

Simple (zero-shot) - intent acc: 0.87 | action acc: 0.77

Agent RAG - intent acc: 0.90 | action acc: 0.67

LLM judge means - overall: 4.39, groundedness: 4.11, helpfulness: 4.39, tone: 4.83, safety: 4.94

Judge vs human - exact: 0.47, within+/-1: 1.00, Cohen kappa: 0.07 (n=17)

4. Failure analysis (top 5 with hypotheses)

Vague Tweets (Just tagging @AmazonHelp):

- **The Problem:** Tweets with no detailed customer question give the AI nothing to act on, causing it to generate generic or guessed replies.
- **Why it happens:** The system is missing the context of the original parent tweet or full conversation thread.

Sarcasm ("Great job losing my package!"):

- **The Problem:** The AI takes sarcastic complaints literally because the words sound positive ("Great job").
- **Why it happens:** Short tweets don't contain enough emotional cues for the model to reliably detect sarcasm.

Multiple Issues in One Tweet (Billing + Shipping):

- **The Problem:** If a customer asks about a shipping delay *and* a refund in the same tweet, the system gets confused.
- **Why it happens:** The system forces every tweet into a single category, so it has to ignore one of the customer's issues.

Unhelpful Search Results (Bad RAG Matches):

- **The Problem:** The search engine retrieves past historical replies that match the words, but are too generic to help write a good answer.
- **Why it happens:** Text similarity in vector search does not always mean the retrieved reply contains a good resolution.

Over-Escalating Simple Queries:

- **The Problem:** The AI transfers simple FAQ questions to human agents whenever it spots trigger words like "refund."
- **Why it happens:** Without access to a live customer account database, the prompt is intentionally programmed to play it safe—preferring unnecessary human hand-offs over making a wrong promise to the customer.

5. What is misleading about my headline number?

Grading AI against AI: The accuracy looks artificially high because the agent is currently being tested against the AI's own "best guesses" (silver labels). True accuracy will only be known once humans review every single label.

The grader is biased: We used the same AI model family (Groq) to both write the replies and grade them. The AI naturally likes its own writing style, which can unfairly boost the reply quality scores.

Oversimplified categories: Sorting customer issues into just 5 broad buckets hides complexity. If a tweet has multiple problems (like a refund *and* a shipping issue), our simple scoring misses those nuances.

Small test sizes: To stay within the free-tier API limits, we default to testing only 30 examples instead of the full 200. This small sample size makes the final percentage less stable and reliable.

6. What I'd do with one more week

If given another week to build on this project, I would focus on:

- **Expanding Human Labels:** Manually review all 200 golden set items and increase human-evaluated reply scores to 50+ samples for stronger validation.
- **Adding a Second AI Judge:** Use a different, independent LLM (such as Claude or GPT-4) as an evaluator to remove single-model bias during scoring.
- **Improving Context:** Include the parent tweet/conversation thread so the agent understands vague customer messages without guessing.
- **Building a Local Intent Classifier:** Train a lightweight, open-source model (like DistilBERT) to handle intent classification locally, reducing reliance on API calls and saving quota limits.
- **Deploying an Interactive Demo:** Build a simple Streamlit user interface to allow real-time testing and manual reviewing of agent responses.

7. Decision log (15 non-obvious choices)

1. **Target Brand Selection (@AmazonHelp):** Chosen for its dense, repetitive transactional support patterns, which are ideal for RAG retrieval grounding.
2. **LLM Provider (Groq [openai/gpt-oss-120b](#)):** Switched to Groq for fast, reproducible inference.
3. **Structured Outputs:** Used OpenAI-compatible JSON Object mode to guarantee predictable, parseable agent responses.
4. **Zero-Cost Embeddings ([all-MiniLM-L6-v2](#)):** Used ChromaDB's default local embedding model to eliminate embedding API costs and rate limits.
5. **Client-Side API Throttling:** Implemented a mandatory ~2.2s gap between requests to strictly respect free-tier rate limits.
6. **Configurable Evaluation Limits:** Set lightweight defaults ([n=30](#) evaluation, [n=20](#) judge) that can be easily scaled up via [.env](#) variables for full runs.
7. **Silver-to-Gold Dataset Pipeline:** Used AI-generated "silver" labels followed by manual human verification to build a 200-sample golden set efficiently.
8. **Intent-Balanced Sampling:** Sampled evaluation rows evenly across intent categories to prevent majority-class bias.
9. **Prompt-Encoded Safety Rules:** Embedded escalation logic directly into system prompts due to the lack of live backend order databases.
10. **Single-Turn Scope:** Focused strictly on direct Question-Answer tweet pairs, leaving multi-turn thread tracking out of the MVP.
11. **5-Dimension LLM Rubric:** Evaluated agent drafts across overall score, groundedness, helpfulness, tone, and safety.

- 12. Standardized Rating Schema:** Created an explicit `human_reply_ratings.csv` contract to prove LLM-vs-human judge alignment.
- 13. MLflow Tracking:** Logged all pipeline variants side-by-side for clear performance comparison across trivial, zero-shot, and RAG setups.
- 14. Conservative Safety Bias:** Designed the agent to prefer unnecessary human hand-offs (false-positive escalations) over risking automated errors on billing/financial queries.
- 15. Transparent Methodology:** Authored a standalone `GOLDEN_SET_METHODOLOGY.md` file so reviewers can easily audit data labeling and evaluation standards

8. Manual setup (free tier) & Reproduction

A. Reviewer Fast-Track (Run Evaluation Only) Since the database and golden set are already included in the repository, you can evaluate the agent immediately:

1. Set your own `GROQ_API_KEY` in your `.env` file. I have included an `.env` example file in Repo.
2. Activate a virtual environment `venv` and install dependencies: `pip install -r requirements.txt`
3. Run the evaluation: `python evaluation.py`
4. View metrics in the terminal or run `mlflow ui` to view the side-by-side comparison.

B. Full Pipeline Reproduction (Optional) If you wish to rebuild the vector database and evaluation sets from scratch (typically this can take more time):

1. Set your own `GROQ_API_KEY` and `KAGGLE_TOKEN` in your `.env` file. I have included an `.env` example file in Repo.
2. Activate a virtual environment `venv` and install dependencies: `pip install -r requirements.txt`
3. Download data from kaggle: `python download_data.py`
4. Process data and build ChromaDB: `python data_processing.py` & `python rag_pipeline.py`
5. Generate new silver labels: `python golden_set_generator.py` , it will generate golden set csv file in data folder.
6. Verify `golden_set_generator.csv` manually and add human intent/actions wherever needed the AI agent will prioritize those.
7. Run the evaluation: `python evaluation.py` , it will generate `llm_judge_scores.csv` file in data folder.
8. Copy the `customer_text` column from `llm_judge_scores.csv` to a new csv file `human_reply_ratings.csv` and provide Human ratings . This is necessary to evaluate llm on human basis and compare AI response .
9. Again run the evaluation ,`python evaluation.py` this time it will log the scores and metrics on console and in MLflow ui.
10. (optional) Run `MLflow ui` for comparison of different Results and baselines.
11. Generate this report: `python generate_report_pdf.py`